

Java Iterators

Traversing Collections with Precision

codehive

codehive

The Iterator Interface

An **Iterator** is an object that can be used to loop through collections, like `ArrayList` and `HashSet`. It is called an "iterator" because "iterating" is the technical term for looping.

```
import java.util.ArrayList;
import java.util.Iterator;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> cars = new ArrayList<>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");

        // Get the iterator
        Iterator<String> it = cars.iterator();

        // Loop through the list
        while(it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

Essential Iterator Methods

The Iterator interface provides three primary methods that allow you to navigate and modify a collection safely:

hasNext()

Returns `true` if the collection has more elements that have not yet been visited.

next()

Returns the next element in the collection and moves the "cursor" forward.

remove()

Removes the last element returned by the iterator from the collection. *Note: This is the safest way to remove items while looping!*

Why use Iterators?

While a `for-each` loop is simpler, Iterators are necessary when you need to **modify the collection** while traversing it. Attempting to remove items from a list using a standard `for` loop often results in a `ConcurrentModificationException`.

Collection Traversal with

codehive

Safe navigation, simple logic.

codehive